PROJECT DOCUMENTATION

MULTI-LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM

USING SALESFORCE

Salesforce | Screen Flow | Approval Process | Apex | LWC

TEAM DETAILS

TEAM ID	SWTID-2026-8095
TEAM SIZE	4
TEAM LEADER	S. Nithish NMID: 13B27FAE19C56F09F3543B6453644851 Register No: 821923243037
TEAM MEMBER	G. Anish Pradeep NMID: E588806BA58D395989045D185F8FC289 Register No: 821923243007
TEAM MEMBER	S. Johnson Infant NMID: B79E6A8BC3205E4B2EA407B0225A3E24 Register No: 821923243023
TEAM MEMBER	S. Bharathraj NMID: 55E53093039601EBC7E49C15B5D3CF60 Register No: 821923243012

ST. JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY
THANJAVUR
COLLEGE CODE: 8219

Contents

1. Introduction
   - Project Overview
   - Purpose

2. Ideation Phase
   - Problem Statements
   - Empathy Map
   - Brainstorming and Idea Prioritization

3. Requirement Analysis
   - Customer Journey Map
   - Solution Requirements
   - Data Flow
   - Technology Stack

4. Project Design
   - Problem-Solution Fit
   - Proposed Solution
   - Solution Architecture

5. Project Planning and Scheduling
   - Planning Approach
   - Product Backlog and Sprint Schedule
   - Sprint Tracker and Velocity

6. Project Development
   - Milestone 1: Salesforce Developer Org
   - Milestone 2: Custom Objects
   - Milestone 3: Record Types and Field Sets
   - Milestone 4: Fields and Relationships
   - Milestone 5: Auto Quoting Screen Flow
   - Milestone 6: Validation Rule
   - Milestone 7: Premium Calculation with Apex
   - Milestone 8: Claim Routing Flow
   - Milestone 9: ClaimsAdjusterController
   - Milestone 10: claimsDashboardLwc
   - Milestone 11: claimTileLwc
   - Milestone 12: Approval Process
   - Milestone 13: Submit-to-Approval Flow
   - Milestone 14: Approver Screen Flow and Quick Action
   - Milestone 15: State Update Flow and Sharing Rules
   - Milestone 16: Permission Sets
   - Milestone 17: Apex Test Class

7. Functional and Performance Testing

8. Advantages and Disadvantages

9. Conclusion

10. Future Scope

11. Appendix

12. Demo Link

1. Introduction

Project Overview

Insurance companies handle several product lines at the same time, and each line asks for different information. When quoting, policy issuance and claims live in registers, spreadsheets and separate tools, quotes take too long, data is entered inconsistently and claims wait in the wrong hands. The Multi-Line Insurance Policy and Claims Management System brings Vehicle, Property and Life insurance onto one Salesforce platform, so every customer, policy and claim sits in a single connected data model.

The application is built in a Salesforce Developer Edition org. The Policy and Claim custom objects use Record Types and Field Sets to hold product-specific data. A guided Screen Flow handles auto quoting, a validation rule keeps VINs clean and Apex calculates the premium. When a claim is created, a Record-Triggered Flow routes it to the Auto, Property or Life queue. Claims above $50,000 go through a two-step approval (Senior Adjuster, then Manager), and adjusters see their own workload in an LWC Claims Dashboard. Access is controlled with three permission sets and state-based sharing rules.

Purpose

- Centralize Vehicle, Property and Life policies and claims in one Salesforce platform.
- Standardize and speed up policy quoting and issuance with a guided Screen Flow.
- Calculate premiums automatically from policy details such as state and model year.
- Route every new claim to the right queue based on its policy type.
- Send high-value claims through a structured, multi-level approval.
- Give claims adjusters a focused dashboard of their assigned claims with amount, status and days open.
- Give each type of user only the access that their job needs, including state-based access.
- Provide a 360-degree view of customer, policy and claim information.

2. Ideation Phase

Problem Statements

The table below records the problems that motivated the project, written from the point of view of each person affected.

#	I am	I am trying to	But	Because	Which makes me feel
PS-1	Insurance Operations Manager	Standardize policy quoting and issuance	The process is manual, slow and inconsistent	Legacy systems and processes are fragmented	Frustrated
PS-2	Insurance Agent	Create and manage policies efficiently	Different insurance lines need different information	Vehicle, Property and Life policies have diverse data	Overwhelmed
PS-3	Insurance Agent	Provide accurate policy quotes	Quoting needs multiple steps and manual processing	There is no unified quoting platform	Uncertain

#	I am	I am trying to	But	Because	Which makes me feel
PS-4	Claims Manager	Manage claims efficiently	Claim review and routing are largely manual	Claim processes are not centralized or automated	Stressed
PS-5	Claims Adjuster	Manage assigned claims effectively	I lack a focused view of my workload	Claim information is not in one place	Less productive
PS-6	Claims Manager	Route claims to the right team	Claims must follow policy type and severity	Routing is complex and needs automation	Concerned
PS-7	Senior Adjuster	Process high-value claims quickly	Approval needs several levels of review	High-value claims need structured escalation	Pressured
PS-8	Insurance Operations Manager	Keep policy information accurate	Products need different fields and checks	Policy types have varying data rules	Concerned
PS-9	Customer	Get my policy and claim handled quickly	Manual processing causes delays	Policy and claim processes are fragmented	Dissatisfied
PS-10	Insurance Management	Get a complete view of customer, policy and claim	Information is spread across processes	There is no unified platform	Concerned

Summary of needs. Manual quoting, different data for each product, slow claim assignment, unreviewed high-value claims and uncontrolled access all point to one need: a single connected system that guides quoting, calculates premiums, routes claims, escalates large claims for approval and limits access by role.

Empathy Map

Persona: Insurance Claims Adjuster. The map captures what the adjuster experiences while handling claims.

THINKS & FEELS Concerned about claim delays and workload	**HEARS** Customer and team concerns about claim resolution	**SEES** Many claims, policy details and approval statuses
SAYS & DOES Reviews claims and follows up on approvals	**PAINS** Manual processing, complex routing, limited visibility	**GAINS** Claims dashboard, automation and faster approvals

Brainstorming and Idea Prioritization

Four approaches were considered: manual registers, spreadsheet tracking, a standalone portal, and a Salesforce CRM. Ideas were then placed on an impact-versus-feasibility grid.

Idea	Approach	Key limitation
Manual registers	Policy and claim information kept by hand	Slow, error-prone and hard to maintain
Spreadsheet tracking	Policies, claims and approvals tracked in sheets	Limited automation and no real-time visibility
Standalone portal	A separate portal for policies and claims	High development and maintenance effort

Idea	Approach	Key limitation
Salesforce CRM (chosen)	One platform for policies, claims, automation and reporting	Needs careful configuration, which this project provides

	LOW FEASIBILITY	HIGH FEASIBILITY
HIGH IMPACT	**High impact, low feasibility** • Real-time external vehicle / property database checks • AI-assisted claim assessment	**High impact, high feasibility** • Guided Auto Quoting Screen Flow • Automatic premium calculation (Apex) • Claim routing by policy type • Two-step approval for high-value claims • Adjuster Claims Dashboard (LWC)
LOW IMPACT	**Low impact, low feasibility** • Separate mobile app for agents	**Low impact, high feasibility** • Custom tab for Claim • Approve/Reject quick action • VIN length validation

Top prioritized ideas: guided quoting, automatic premium, claim routing, high-value claim approval and the adjuster dashboard.

Final Selection

A Salesforce CRM was chosen because it offers:

- Automation for quoting, premium calculation, routing and approvals
- Scalability for Vehicle, Property, Life and future insurance lines
- Real-time visibility on one unified platform
- Built-in security with permission sets and sharing rules
- Less manual processing for agents and claims users

3. Requirement Analysis

Customer Journey Map

A customer moves through twelve stages, from the first insurance enquiry to reporting on the final claim position.

Stage	Customer / user action	System action	Outcome
1. Insurance inquiry	Customer asks about cover	Agent opens the quoting flow	Enquiry started
2. Policy selection	Chooses Vehicle, Property or Life	Matching Record Type is selected	Product chosen
3. Quote request	Requests a quote	Auto Quoting Screen Flow starts a guided quote	Quote started
4. Information capture	Gives holder, start date, state, VIN and model year	Details are entered; VIN length is validated	Clean policy data
5. Premium calculation	No direct action	PremiumCalculator Apex returns the premium	Premium stored
6. Policy issuance	Receives the policy	Validated policy is stored in Policy__c	Policy issued
7. Claim initiation	Raises a claim	Claim__c record is created and linked to the policy	Claim recorded
8. Claim routing	No direct action	Record-Triggered Flow assigns the Auto, Property or Life queue	Right queue

Stage	Customer / user action	System action	Outcome
9. Review and approval	Waits for the decision	Claims above $50,000 go to the Senior Adjuster, then the Manager	Controlled review
10. Claim resolution	No direct action	Approval decision updates the Approval Status field	Claim decided
11. Customer confirmation	Is told the outcome	Claims team informs the customer	Customer informed
12. Reporting and visibility	No direct action	Claims Dashboard and reports show the live position	Clear overview

Solution Requirements

Functional requirements

FR No	Requirement
FR-01	Create and manage Insurance Policy records for Vehicle, Property and Life insurance
FR-02	Support separate Record Types for Vehicle, Property and Life policies
FR-03	Capture product-specific data such as VIN for Vehicle and Square Footage / Year Built for Property
FR-04	Provide a guided Auto Quoting Screen Flow capturing policy holder, start date, state, VIN and model year
FR-05	Create a draft policy during the quoting process
FR-06	Calculate policy premiums with Apex-based logic
FR-07	Validate that a Vehicle VIN contains exactly 17 characters
FR-08	Create and manage Claim records associated with policies
FR-09	Support Claim Record Types for Accident, Property and Life claims
FR-10	Route claims automatically to the right queue based on the related policy type
FR-11	Identify high-value claims using the configured amount threshold
FR-12	Send qualifying claims through a multi-level approval (Senior Adjuster, then Claims Manager)
FR-13	Provide Approve/Reject with approver comments
FR-14	Provide Claims Adjusters an LWC Claims Dashboard of assigned claims
FR-15	Show Claim Number, Policy Type, Policy Holder, Claim Amount, Status and Days Open
FR-16	Allow filtering of claims by All, Auto, Property and Life
FR-17	Update related Policy and Claim data through Record-Triggered Flows
FR-18	Support state-based access and sharing
FR-19	Provide role-specific Permission Sets for Agents, Adjusters and Managers
FR-20	Support reporting and visibility of customer, policy and claim information

Non-functional requirements

Attribute	Description
Usability	Simple guided Screen Flows cut unnecessary clicks for agents
Performance	Quoting, premium calculation, routing and dashboard actions run efficiently

Attribute	Description
Scalability	Supports Vehicle, Property and Life now and more products later
Data integrity	Policy and claim data is validated so records stay accurate and consistent
Security	Access follows responsibility, with state-based sharing and Permission Sets
Maintainability	Modular Flows, Apex, LWC, Record Types and Field Sets can be changed independently
Reliability	Routing, premium calculation and approvals run consistently by the configured rules
Extensibility	Product-specific fields fit the same structure without a new model per line
Visibility	Timely access to customer, policy and claim data through records and the dashboard
User adoption	An intuitive experience that reduces manual effort for agents and claims users

Data Flow

From the agent's input to the final dashboard, data moves through the following stages:

Agent input	Auto Quoting Screen Flow	VIN validation	Policy__c (draft)	Premium Calculator
Claim__c created	Routing Flow	Claims Queue	Approval above $50,000	Adjuster Dashboard

Approval path: Claim Amount greater than 50000 > Submission Automation Flow > Step 1 Senior Adjuster > Step 2 Claims Manager > Approved or Rejected > Approval Status updated.

Step	What happens
1. Input	The agent provides customer and policy information
2. Capture	The AutoQuoting Screen Flow captures policy holder, start date, state, VIN and model year
3. Validation	Salesforce validates the data, including the 17-character VIN rule for Auto policies
4. Policy creation	Valid data is stored in Policy__c under the Auto, Property or Life Record Type
5. Premium	PremiumCalculator Apex calculates the premium from the policy data
6. Claim creation	A Claim__c record is created and linked to the policy
7. Routing	A Record-Triggered Flow reads the policy type and routes the claim to the right queue
8. Review	The adjuster opens assigned claims in the Claims Dashboard
9. Approval	Claims above the threshold go to the Senior Adjuster, then the Claims Manager
10. Decision	The approver chooses Approve or Reject with comments and the status updates
11. Reporting	The dashboard shows claim amount, policy type, holder, status and days open

Technology Stack

Layer	Technology
Platform	Salesforce CRM, Developer Edition org
Data	Custom objects Policy__c and Claim__c; Record Types; Field Sets; Lookup relationships to Contact and User
Logic	Apex classes PremiumCalculator, ClaimsAdjusterController and its test class (API version 67)

Layer	Technology
Automation	Screen Flow, Record-Triggered Flows, Approval Process, Field Updates, Validation Rule
UI	Lightning pages, Lightning Web Components (claimsDashboardLwc, claimTileLwc), Quick Action
Security	Permission Sets and state-based Sharing Rules
Insight	Claims Dashboard, Salesforce Reports and Dashboards

4. Project Design

Problem-Solution Fit

Problem	Solution	Benefit
Manual, inconsistent quoting	Auto Quoting Screen Flow	Guided, standard quotes
Different data per product	Record Types and Field Sets	One model, many products
Wrong VINs entered	VIN_Must_Be_17_Characters validation rule	Clean vehicle data
Manual premium work	PremiumCalculator Apex	Consistent premiums
Slow claim assignment	Claim routing Record-Triggered Flow	Right queue instantly
High-value claims unreviewed	Two-step approval process and flows	Controlled escalation
No adjuster workload view	claimsDashboardLwc and claimTileLwc	Focused daily view
Different access needs	Permission Sets and state sharing rules	Secure, role-based access

Proposed Solution

Parameter	Description
Problem	Quoting, policy issuance and claims live in registers, spreadsheets and separate tools, so quotes are slow, data is inconsistent and claims wait in the wrong hands.
Solution	A Salesforce CRM that stores customers, policies and claims for Vehicle, Property and Life insurance in one connected data model.
Innovation	A guided Screen Flow, Apex premium calculation, automatic claim routing, a two-step approval and an LWC adjuster dashboard replace manual steps.
Impact	Faster, standard quotes, fewer data errors, claims in the right hands and a clear view of the live position.
Business model	Insurance companies working with agents, adjusters, claims managers and customers across several product lines.
Scalability	Scales through Record Types, Field Sets and modular automation, with room for more products, external integrations and customer-facing tracking.

Solution Architecture

The solution is organized in layers. Users work through Lightning pages, flows and the dashboard, automation and Apex apply the business rules, and Salesforce records hold the data.

Users	Insurance Agent | Claims Adjuster | Claims Manager
UI layer	Lightning Pages | Screen Flows | LWC Dashboard

Automation layer	Record-Triggered Flows | Approval Process | Validation Rule
Apex layer	PremiumCalculator | ClaimsAdjusterController | ClaimsAdjusterControllerTest
Data layer	Policy__c (3 Record Types) | Claim__c (3 Record Types) | Contact | User
Security and insight	Permission Sets | State-based Sharing Rules | Reports and Dashboards

Data model

Object	Key fields	Links
Policy__c	Policy Number (P-0000), VIN, Model Year, Square Footage, Year Built, Beneficiary Name, Policy Term Months, Premium, Policy State	Customer (Lookup to Contact)
Claim__c	Claim Number (C-0000), Claim Amount, Approval Status, Date of Loss, Description, Policy Account Holder State	Policy (Lookup to Policy__c); Adjuster (Lookup to User)

5. Project Planning and Scheduling

Planning Approach

- Agile methodology with sprint-based execution
- Work is split into epics and user stories and sized in story points
- Velocity is calculated from completed story points and used to check progress

Product Backlog and Sprint Schedule

Sprint	Epic	USN	User story / task	Pts	Priority	Owner
1	Data Modeling	US-01	As an admin, I want to create Policy and Claim objects so that policy and claim information can be stored and managed systematically.	5	High	S. Nithish
1	Data Modeling	US-02	As an admin, I want to configure Vehicle, Property and Life Record Types so that different products can be managed in one Policy structure.	3	High	G. Anish Pradeep
1	Data Modeling	US-03	As an admin, I want to configure policy Field Sets and product-specific fields so that diverse insurance information is captured efficiently.	5	High	S. Johnson Infant
1	Quoting and Automation	US-04	As an insurance agent, I want a guided Auto Quoting Screen Flow so that policy information is captured in a standard way.	8	High	S. Bharathraj
2	Validation	US-05	As an insurance agent, I want VIN validation during vehicle quoting so that incorrect vehicle data is prevented.	3	High	S. Nithish
2	Apex	US-06	As an insurance system, I want PremiumCalculator Apex to calculate premiums so that premium values are accurate.	8	High	G. Anish Pradeep

Sprint	Epic	USN	User story / task	Pts	Priority	Owner
2	Automation	US-07	As an insurance agent, I want the premium calculation integrated with the quoting Flow so that the premium is available during the quote.	5	High	S. Johnson Infant
2	Claims Automation	US-08	As a claims manager, I want claims routed by policy type so that they reach the right support queue.	8	High	S. Bharathraj
3	Apex	US-09	As a claims adjuster, I want a ClaimsAdjusterController to retrieve my assigned claims so that I can manage my workload.	8	High	S. Nithish
3	LWC Dashboard	US-10	As a claims adjuster, I want a claims dashboard and reusable claim tile so that I can view and manage assigned claims.	8	High	G. Anish Pradeep
4	Approval	US-11	As a claims manager, I want high-value claims to go through an approval process and approver Flow so that they get proper review.	8	High	S. Johnson Infant
4	Security and Testing	US-12	As a system administrator, I want tests, sharing rules and Permission Sets so that the solution is validated and users get the right access.	8	High	S. Bharathraj

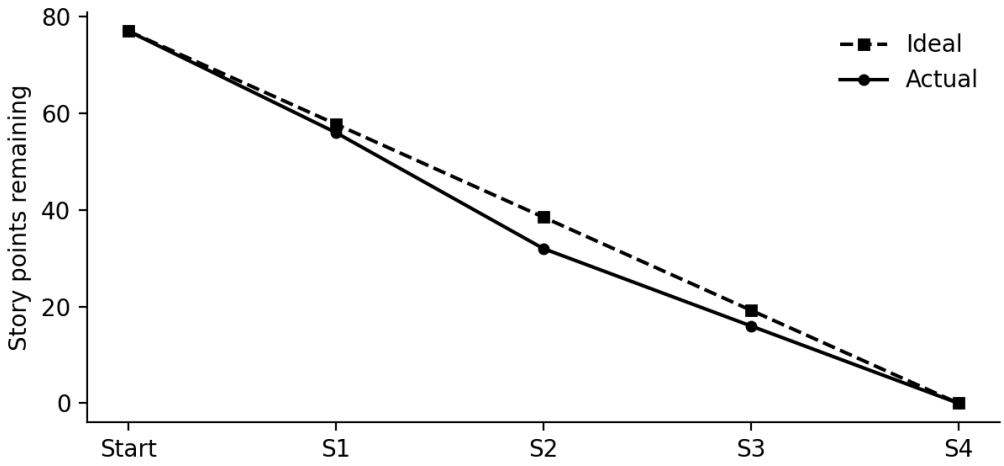
Story-point total: 77 across four sprints (21 + 24 + 16 + 16).

Sprint Tracker and Velocity

Sprint	Story points	Stories	Points completed	Duration
Sprint-1	21	US-01 to US-04	21	6 days
Sprint-2	24	US-05 to US-08	24	6 days
Sprint-3	16	US-09, US-10	16	6 days
Sprint-4	16	US-11, US-12	16	6 days
Total	**77**	**12 user stories**	**77**	**24 days**

Average velocity = total story points / number of sprints = 77 / 4 = about 19.3 points per sprint. Sprint 2 carried the most work (24 points) because the PremiumCalculator Apex logic, its Flow integration and the claim routing flow were built together.

Sprint Burndown (story points remaining)



Sprint burndown: ideal (dashed) versus actual (solid).

6. Project Development

The topics covered while building the solution were org setup, data modeling with Record Types, Screen Flow and Record-Triggered Flows, validation, Apex, Lightning Web Components, the approval process and the Salesforce security model.

Milestone 1: Salesforce Developer Org

A Salesforce Developer Edition org was signed up for using first name, last name, email, role (Developer), company (college name), country (India), postal code and a unique username. The account was verified from the email link, a password and security question were set, and Setup was opened. All objects, automation and code in this project were built inside this org.

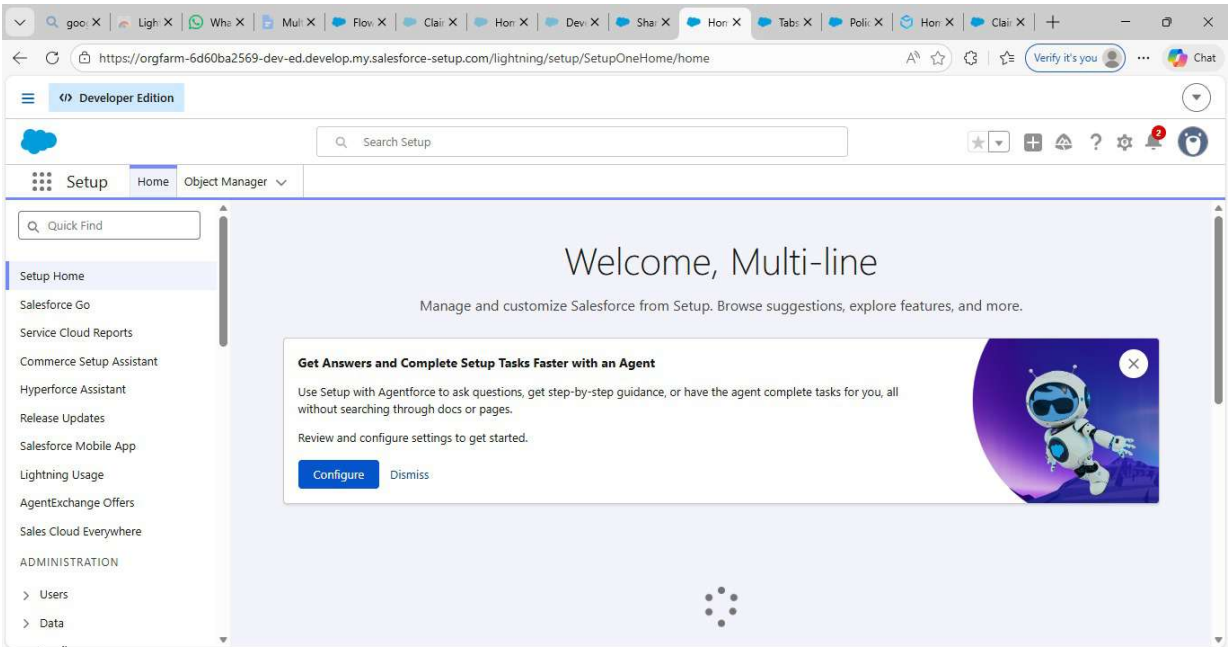


Figure 1. Salesforce Setup home of the Developer Edition org used for this project.

Milestone 2: Custom Objects

Two custom objects were created from Setup > Object Manager > Create > Custom Object. For both, Allow Reports, Allow Activities and Track Field History were enabled, the object was set to Deployed

and Allow Search was checked.

Setting	Policy	Claim
Label / Plural label	Policy / Policies	Claim / Claim
Record name	Policy Number	Claim Number
Data type	Auto Number	Auto Number
Display format	P-{0000}	C-{0000}
Starting number	1	1

A custom object tab was also created for Claim (Setup > Tabs > Custom Object Tabs > New) so that users can open claim records quickly.

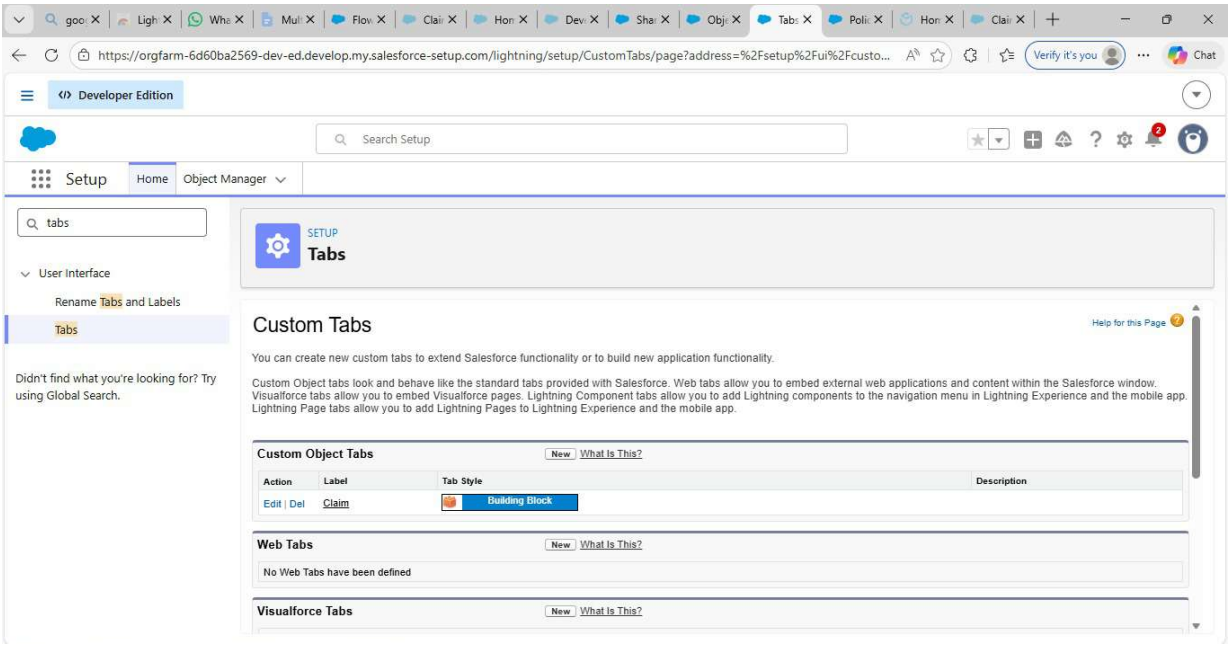


Figure 2. Custom Object Tabs in Setup showing the Claim tab.

Milestone 3: Record Types and Field Sets

Record Types let one Policy object serve three insurance lines. On Policy__c, the Record Types **Auto**, **Property** and **Life** were created from Object Manager > Record Types > New (based on Master, active, available to all profiles, default page layout). The same was done on Claim__c for **Accident**, **Property** and **Life**.

Field Sets group the fields that belong to each product. They were created from Object Manager > Policy > Field Sets > New:

Field Set	Fields included
Vehicle	VIN, Model Year
Property	Square Footage, Year Built
Life	Beneficiary Name, Policy Term Months

Milestone 4: Fields and Relationships

Policy object fields

#	Field name	Data type
1	Beneficiary Name	Text (50)
2	Customer	Lookup (Contact)
3	Model Year	Text (5)
4	Policy Start Date	Date
5	Policy Term Months	Number (18, 0)
6	Premium	Currency (16, 2)
7	Square Footage	Number (18, 0)
8	VIN	Text (20)
9	Year Built	Text (5)
10	Policy State	Picklist (used by the quoting flow, premium logic and sharing)

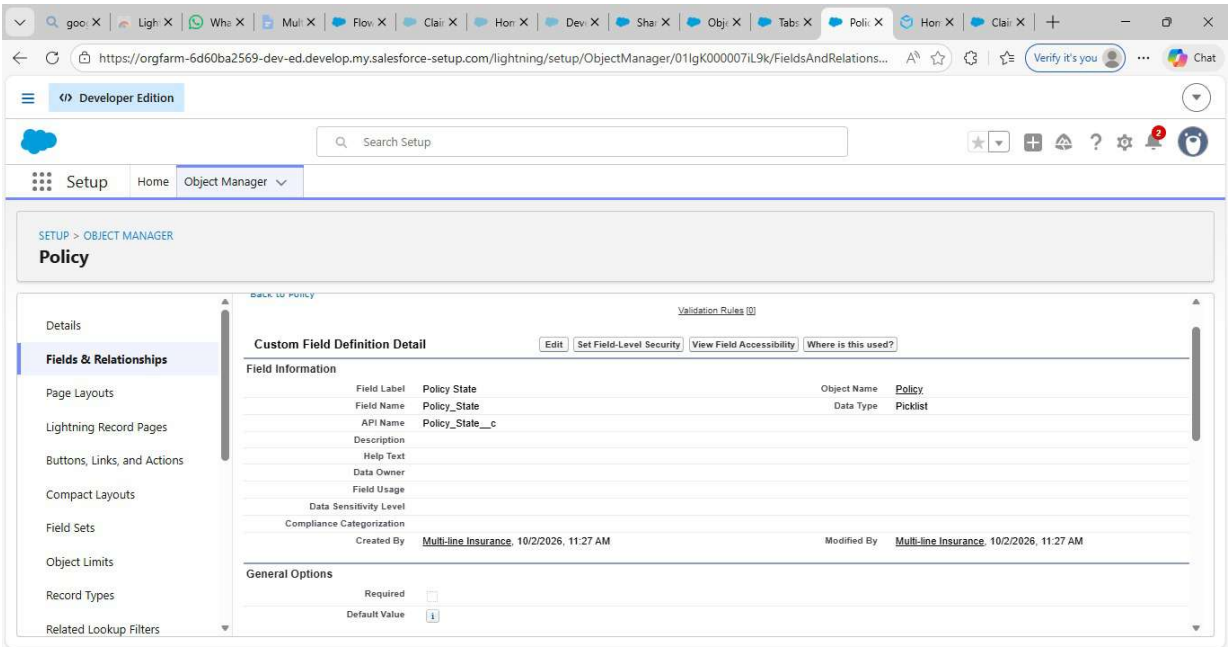


Figure 3. Policy object: Policy State picklist field definition (API name Policy_State__c).

Claim object fields

#	Field name	Data type
1	Adjuster	Lookup (User)
2	Approval Status	Picklist (New, Submitted for Approval, Approved, Rejected)
3	Claim Amount	Currency (16, 2)
4	Date of Loss	Date/Time
5	Description	Long Text Area
6	Policy	Lookup (Policy)
7	Policy Account Holder State	Text (filled by the state-update flow)

Relationship	Type	Purpose
Policy to Contact (Customer)	Lookup	Each policy belongs to a customer

Relationship	Type	Purpose
Claim to Policy	Lookup	Each claim is raised against a policy
Claim to User (Adjuster)	Lookup	Each claim is handled by an adjuster

Milestone 5: Auto Quoting Screen Flow

The flow **AutoQuotingFlow** (Setup > Flows > New Flow > Screen Flow) walks an agent through a quote in four elements.

Element	Configuration
Get Records: Get Vehicle RT ID	Object: Record Type. Conditions (all met): sObject Type Name equals Policy__c and Record Type Name equals Auto.
Screen: New Auto Policy Quote - Basic Information	Lookup component Policy Holder (Customer__c on Policy__c), Date component Policy Start Date, Picklist component Policy State (picklist choice set on Policy > Policy State).
Screen: Vehicle-Specific Details	Text component VIN and text component Model year.
Create Records: Create Draft Policy	Policy object with Customer, Model Year, Policy Start Date, Policy State, VIN and RecordTypeId set from the screens and from Get Vehicle RT ID. The new Id is stored in the variable varPolicyId.

After Milestone 7, the flow was extended after the Create Records element. The action **Calculate Premium Action** calls PremiumCalculator with varPolicyId and stores the output in the currency variable varCalculatedPremium. **Update Policy With Premium** then finds the Policy where Record Id equals varPolicyId and sets Premium to varCalculatedPremium. The flow was saved as a new version and activated.

Milestone 6: Validation Rule

To stop incorrect vehicle data, the Policy object has an active validation rule **VIN_Must_Be_17_Characters**. It applies to the Auto Record Type, shows its message on the VIN field and blocks the save when the VIN length is not 17.

Error condition formula

```
AND( RecordType.DeveloperName = "Auto", LEN( VIN__c ) <> 17 )
```

Error message: *The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.*

Milestone 7: Premium Calculation with Apex

The Apex class **PremiumCalculator** was created in the Developer Console (File > New > Apex Class). It has an @InvocableMethod so that a Flow can call it. The base premium is 1000. It is raised by 15% for California (CA) and 5% for Texas (TX), and 100 is added when the model year is before 2018.

PremiumCalculator.cls

```
public class PremiumCalculator {
  public class PremiumInputWrapper {
    @InvocableVariable(required=true)
    public Id policyId;
  }
  @InvocableMethod(label='Calculate Policy Premium'
    description='Calculates premium based on policy fields (VIN, Model Year, State).')
  public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
    List<Decimal> premiumResults = new List<Decimal>();
    Set<Id> policyIds = new Set<Id>();
    for (PremiumInputWrapper input : inputs) {
      if (input.policyId != null) {
        policyIds.add(input.policyId);
      }
    }
    List<Policy__c> policiesToRate = [
      SELECT Id, VIN__c, Model_Year__c, Policy_State__c
      FROM Policy__c
      WHERE Id IN :policyIds
    ];
    for (Policy__c policy : policiesToRate) {
      Decimal basePremium = 1000.00;
      if (policy.Policy_State__c == 'CA') {
        basePremium *= 1.15;
      } else if (policy.Policy_State__c == 'TX') {
        basePremium *= 1.05;
      }
      if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) < 2018) {
        basePremium += 100;
      }
      premiumResults.add(basePremium);
    }
    return premiumResults;
  }
}
```

Milestone 8: Claim Routing Flow

A Record-Triggered Flow on the Claim object (After Save, run when a record is created, optimized for Actions and Related Records) sends each new claim to the right queue.

Element	Configuration
Get Records: Get_Policy_RT	Policy where Id equals the triggering Claim’s Policy. First record only; the Record Type value is stored in varPolicyRTName (Text).
Decision: Route by Policy Type	Outcome Auto Claim Route when varPolicyRTName equals Auto. Outcome Property Claim Route when it equals Property. Default outcome Life Claim Route.
Update Records (one per path)	Assign to Auto Queue, Assign to Property Queue, Assign to Life Queue. Each updates the Claim whose Id equals the triggering record and sets OwnerId to the 18-character Id of its queue.

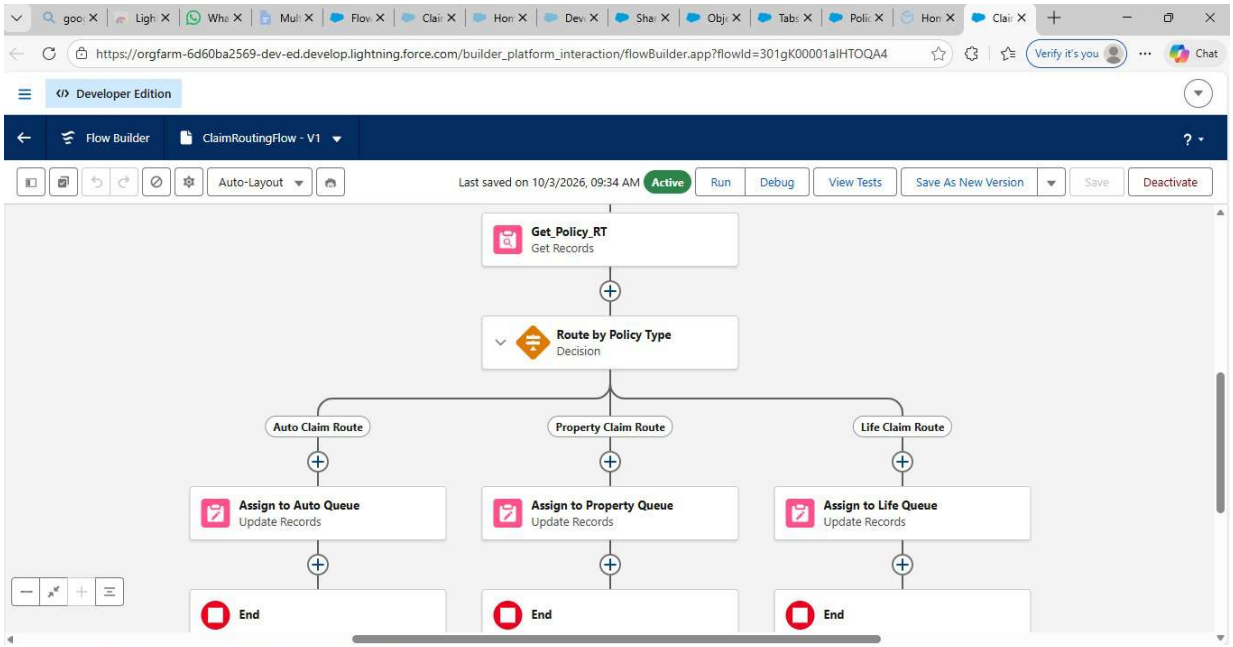


Figure 4. Claim routing flow: Get Records, Decision and three queue assignments.

Milestone 9: ClaimsAdjusterController

The class **ClaimsAdjusterController** runs a SOQL query for all claims owned by the current user, joining the related Policy and Customer data, and returns a list of ClaimWrapper objects with the key metrics for each claim (claim number, amount, status, policy type, policy holder and days open). The method is @AuraEnabled(cacheable=true) so the LWC can call it with @wire.

ClaimsAdjusterController.cls

```
public class ClaimsAdjusterController {
  public class ClaimWrapper {
    @AuraEnabled public Id claimId;
    @AuraEnabled public String claimNumber;
    @AuraEnabled public Decimal claimAmount;
    @AuraEnabled public String status;
    @AuraEnabled public String policyType;
    @AuraEnabled public String policyHolderName;
    @AuraEnabled public Integer daysOpen;
    public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
      this.claimId = c.Id;
      this.claimNumber = c.Name;
      this.claimAmount = c.Claim_Amount__c;
      this.status = c.Approval_Status__c;
      Policy__c policyRecord = (Policy__c)p;
      if (policyRecord != null && policyRecord.RecordType != null) {
        this.policyType = policyRecord.RecordType.Name;
      } else {
        this.policyType = 'Unknown';
      }
      this.policyHolderName = (customerRec != null) ? customerRec.Name : '';
      this.daysOpen = (c.Date_of_Loss__c != null)
        ? (Date.today().daysBetween(c.Date_of_Loss__c.date())) : 0;
    }
  }
  @AuraEnabled(cacheable=true)
  public static List<ClaimWrapper> getAssignedClaims() {
    List<ClaimWrapper> results = new List<ClaimWrapper>();
    List<Claim__c> claims = [
      SELECT Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
        Policy__r.Name, Policy__r.RecordType.DeveloperName,
        Policy__r.Customer__r.FirstName, Policy__r.Customer__r.LastName
      FROM Claim__c
      WHERE OwnerId = :UserInfo.getUserId()
      ORDER BY Date_of_Loss__c DESC
    ];
    for (Claim__c claim : claims) {
      Policy__c policy = claim.Policy__r;
      Contact customer = (policy != null && policy.Customer__r != null)
        ? (Contact)policy.Customer__r : new Contact();
      results.add(new ClaimWrapper(claim, policy, customer));
    }
    return results;
  }
}
```

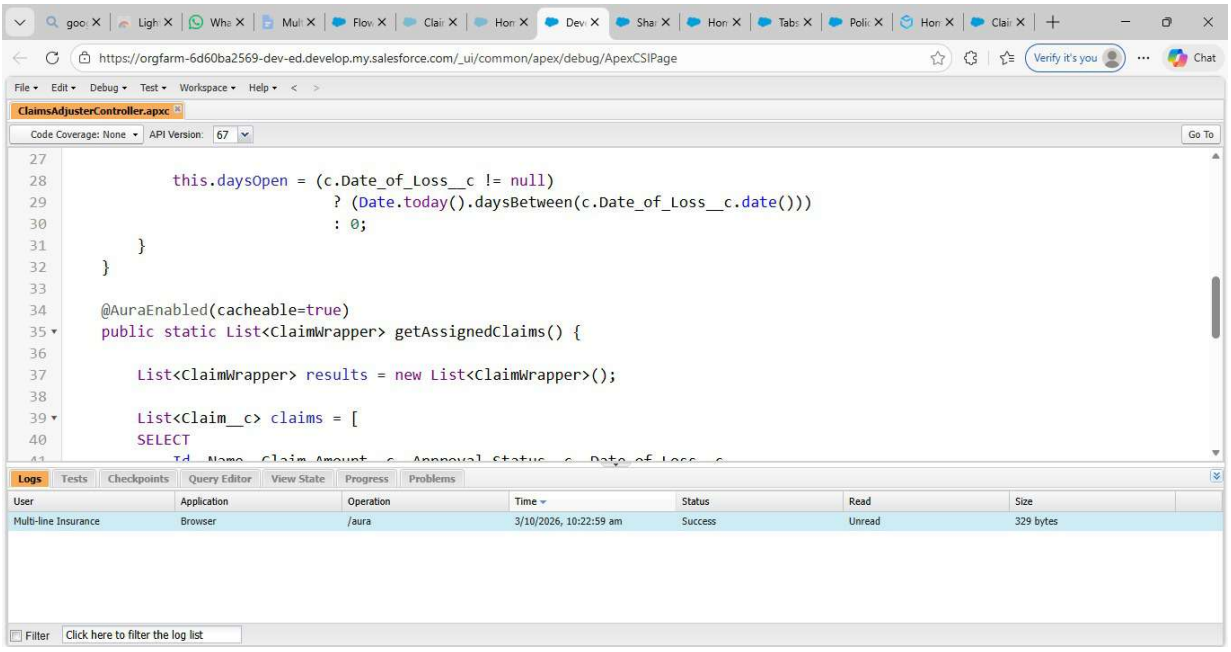


Figure 5. ClaimsAdjusterController saved in the Developer Console.

Milestone 10: claimsDashboardLwc

The component **claimsDashboardLwc** was created in the Lightning Studio browser extension (Create New > Lightning Web Component, isExposed checked, targets Lightning app page and Home page)

and deployed. It loads the claims with @wire, keeps two lists (all claims and visible claims) and filters on the client when the user changes the **Filter by Policy Type** combobox (All, Auto, Property, Life). Each claim is shown with the child component claimTileLwc, and a total count of assigned claims is displayed below the tiles.

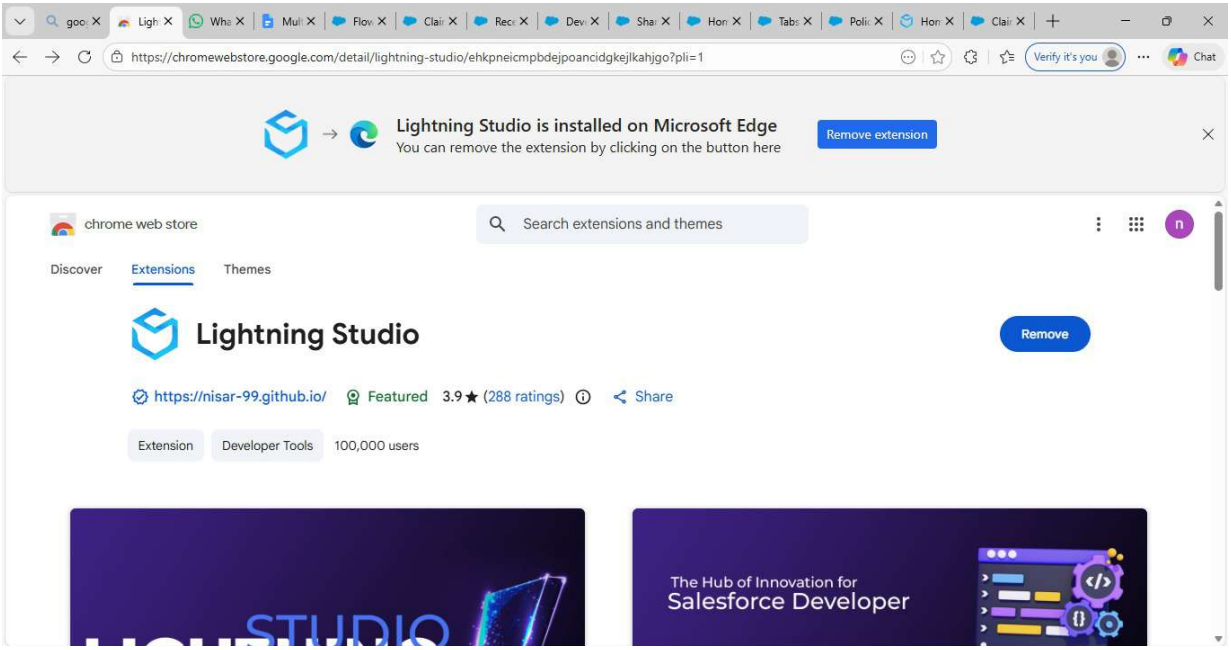


Figure 6. Lightning Studio extension installed in the browser, used to create and deploy the LWC components.

claimsDashboardLwc.js (key logic)

```
import { LightningElement, wire, track } from 'lwc';
import getAssignedClaims from '@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

export default class ClaimsDashboardLwc extends LightningElement {
  @track allClaims = [];
  @track visibleClaims = [];
  error;

  @wire(getAssignedClaims)
  wiredClaims({ error, data }) {
    if (data) {
      this.allClaims = data;
      this.visibleClaims = data;
      this.error = undefined;
    } else if (error) {
      this.error = error;
      this.allClaims = undefined;
      this.visibleClaims = undefined;
    }
  }
  handleFilterChange(event) {
    const selectedPolicyType = event.target.value;
    if (selectedPolicyType === 'All') {
      this.visibleClaims = this.allClaims;
    } else {
      this.visibleClaims = this.allClaims.filter(
        claim => claim.policyType === selectedPolicyType);
    }
  }
  get policyTypeOptions() {
    return [
      { label: 'All Policy Types', value: 'All' }, { label: 'Auto', value: 'Auto' },
      { label: 'Property', value: 'Property' },    { label: 'Life', value: 'Life' }
    ];
  }
}
```

Milestone 11: claimTileLwc

The reusable child component **claimTileLwc** shows one claim: claim number, policy type with an icon (truck for Auto, home for Property, connected apps for Life), policy holder, claim amount as currency, status and days open. The parent passes the data in with an @api claimData property.

claimTileLwc.js

```
import { LightningElement, api } from 'lwc';
export default class ClaimTileLwc extends LightningElement {
  @api claimData;
  get policyIcon() {
    if (this.claimData && this.claimData.policyType) {
      const type = this.claimData.policyType.toLowerCase();
      if (type === 'auto') return 'utility:truck';
      if (type === 'property') return 'utility:home';
      if (type === 'life') return 'utility:connected_apps';
    }
    return 'utility:help';
  }
}
```

Result: Adjusters see only the claims they own, with claim number, policy type, holder, amount, status and days open, and can narrow the view to Auto, Property or Life with one click.

Milestone 12: Approval Process

A picklist field **Approval Status** (New, Submitted for Approval, Approved, Rejected) tracks where each claim stands. The approval process **High Value Claim Approval** was created on the Claim object from Setup > Approval Processes > Create New Approval Process > Use Standard Setup Wizard.

Setting	Value
Entry criteria	Claim Amount greater than 50000
Record editability	Administrators ONLY can edit records during approval
Approval page layout fields	Claim Number, Owner, Claim Amount, Adjuster, Policy
Security	Approvers can access the page only from within Salesforce
Initial submitters	Claim Owner

Step	Configuration
Step 1: Senior Adjuster	All records enter the step. Approver is assigned automatically to the Senior Adjuster user. With several approvers, the decision follows the FIRST response.
Step 2: Manager Review	All records enter the step. Approver is the Department (Claims) Manager user, FIRST response decides. On reject: perform all rejection actions for this step AND all final rejection actions.

Stage	Field update	New value of Approval Status
Initial submission	Update Status Submitted	Submitted for Approval
Final approval	Update Status Approved	Approved
Final rejection	Update Status Rejected	Rejected

Milestone 13: Submit-to-Approval Flow

A Record-Triggered Flow (After Save) on Claim starts the approval automatically. It runs when a record is created or updated, with the entry conditions **Claim Amount > 50000** and **Approval Status = New**, every time the record is updated and meets the conditions, optimized for Actions and Related Records. An Action element **Submit Claim to Approval** passes the triggering Claim’s Record Id and the approval process name *High Value Claim Approval*. The flow is saved as **Submission Automation Flow** and

activated.

Milestone 14: Approver Screen Flow and Quick Action

Approvers can decide from the claim record itself. The Screen Flow **Claim Approver Screen Flow** takes the record Id in a text variable recordId (available for input) and works like this:

- **Get Records: Get Claim Details** finds the Claim whose Id equals recordId and stores all fields.
- **Screen: Claim Review Screen** shows the claim name, amount and policy type as display text, a radio group **Approval Decision** (choices Approve and Reject) and a long text field **Approver Comments**.
- **Decision: Check Decision** has the outcome *Decision Approved* when the radio value equals Approve; the default outcome is *Decision Rejected*.
- Each path runs a **Submit for Approval** action (*Approve Claim* or *Reject Claim*) with recordId and the approver comments.

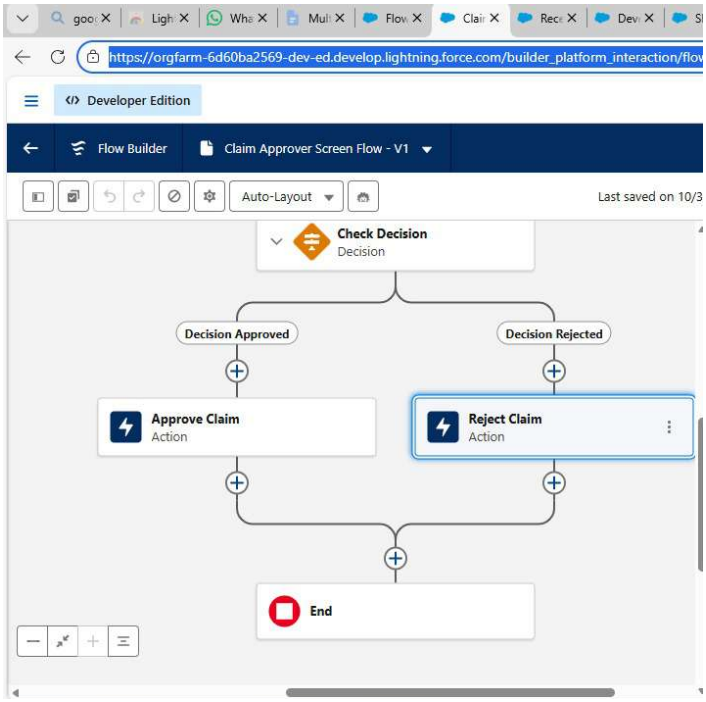


Figure 7. Claim Approver Screen Flow: review screen, decision and the approve and reject actions.

A Quick Action was then created (Object Manager > Claim > Buttons, Links and Actions > New Action): type **Flow**, flow *Claim Approver Screen Flow*, label **Approve/Reject Claim**. It was dragged into Salesforce Mobile and Lightning Experience Actions on the Claim page layout.

Milestone 15: State Update Flow and Sharing Rules

Adjusters should only see claims in their territory. First, a Record-Triggered Flow **Claim Policy Holder State Update** (created or updated, no entry conditions, optimized for Updates) copies the policy state onto the claim: *Get Records get_policy_records* reads the policy state, a *Loop Claim_Filed_update* goes through the policies, and *Update Records Claim_Filed_state_update* stores the loop item's Policy State in the claim field **Policy Account Holder State**.

Then two criteria-based sharing rules were added in Setup > Sharing Settings > Claim Sharing Rules:

Rule label	Criteria	Access
Share Records based on Policy State CA	Policy Account Holder State equals CA	Read/Write

Rule label	Criteria	Access
Share Records based on Policy State	Policy Account Holder State equals TX	Read/Write

Milestone 16: Permission Sets

Three permission sets (Setup > Permission Sets > New) separate what agents, adjusters and managers can do.

Permission set	Object access	Other permissions
Insurance Agent Access	Policy: Create, Read. Contact: Read, Edit. Claim: Read only.	Flow User enabled; PremiumCalculator Apex class enabled.
Claims Adjuster Access	Policy: Read only. Claim: Read, Edit.	ClaimsAdjusterController Apex class enabled.
Claims Manager Access	Policy: Read only. Contact: Read, Edit. Claim: Read, Edit, Delete.	Run Reports and Manage Approvals enabled.

Milestone 17: Apex Test Class

The class **ClaimsAdjusterControllerTest** tests the dashboard controller. A @testSetup method creates a test user, a Contact, an Auto Policy and an Accident Claim, all owned by that user. The test then runs as that user, calls getAssignedClaims() and asserts the claim Id, number, status, amount, policy holder name and policy type. The target is 95% or more code coverage. To run it, open the class in the Developer Console, click **Run Test** and read the coverage in the Tests tab.

7. Functional and Performance Testing

The table lists the checks planned for the main features. The Result column is filled in after each test is run in the org, with a screenshot attached as evidence.

ID	Feature	Test steps	Expected result	Result
TC-01	Auto quoting flow	Run AutoQuotingFlow with a customer, start date, state, VIN and model year.	A draft Auto policy is created.	Pending
TC-02	VIN validation	Save an Auto policy with a 10-character VIN.	Save is blocked with the VIN error message.	Pending
TC-03	Premium calculation	Quote a policy for state CA with model year 2015.	Premium = 1000 x 1.15 + 100 = 1250.	Pending
TC-04	Claim routing (Auto)	Create a claim on an Auto policy.	Claim owner becomes the Auto queue.	Pending
TC-05	Claim routing (Property / Life)	Create claims on a Property and a Life policy.	Claims go to the Property and Life queues.	Pending
TC-06	High-value approval	Create a claim of 60000 with status New.	Claim is submitted; status becomes Submitted for Approval.	Pending
TC-07	Approve / Reject	Use the Approve/Reject Claim action on a submitted claim.	Status becomes Approved or Rejected with comments.	Pending
TC-08	Claims dashboard	Open the dashboard as an adjuster and use the filter.	Only own claims show; filter narrows by policy type.	Pending

ID	Feature	Test steps	Expected result	Result
TC-09	State sharing	Create claims for CA and TX policies.	Policy Account Holder State is filled; sharing rules apply.	Pending
TC-10	Permission sets	Log in as an agent, adjuster and manager.	Each user has only the access of their permission set.	Pending
TC-11	Apex test	Run ClaimsAdjusterControllerTest.	Test passes with 95% or more coverage.	Pending

8. Advantages and Disadvantages

Advantages

- Vehicle, Property and Life policies and claims sit in one centralized Salesforce platform.
- Screen Flows simplify and standardize quoting.
- Apex calculates premiums the same way every time.
- Claims reach the right queue automatically by policy type.
- The approval process reduces manual effort for high-value claims.
- The LWC dashboard gives adjusters a focused view of their workload.
- Validation rules keep policy information correct.
- Record Types and Field Sets fit different products in one flexible model.
- Permission Sets and sharing rules give role-based and state-based access.
- The design is scalable and can take more insurance lines and features.

Disadvantages

- Vehicle, Property and Life policies need different fields, which makes the data more complex to manage.
- Approval must consider claim amount, policy type and adjuster level, which adds complexity.
- The $50,000 approval limit, queue Ids and premium rates are set by hand in the flows and Apex.
- Only the dashboard controller has an Apex test class; PremiumCalculator has no test class yet.
- The state-update flow loops over every policy returned by Get Records instead of reading the claim’s own Policy lookup.
- ClaimsAdjusterController calculates Days Open as Date.today().daysBetween(Date of Loss), which gives a negative value for a past loss date, and it reads RecordType.Name although the query selects RecordType.DeveloperName.
- Simulating external database callouts would need more Apex and mock test classes.
- The solution depends on Salesforce configuration and capabilities.
- It was built in a Developer Edition org for academic purposes and is not deployed to production; a real rollout needs sandboxes, Change Sets or Salesforce DX.

9. Conclusion

The Multi-Line Insurance Policy and Claims Management System shows how a standard Salesforce org can be shaped into a working policy and claims platform. Vehicle, Property and Life insurance data now lives in one place. Guided quoting, automatic premium calculation, claim routing, structured approvals and an adjuster dashboard replace the manual and fragmented steps of the old process.

Through this project the team learned data modelling with Record Types and Field Sets, Screen and Record-Triggered Flows, Apex and test classes, Lightning Web Components, approval processes and the Salesforce security model. The outcome is faster policy processing, better claim handling, cleaner data, secure role-based access and a clear operational view for insurance teams.

10. Future Scope

- **External integration:** connect to external vehicle and property databases for real-time quote verification, with API-based integrations to insurance and verification systems.
- **More insurance products:** extend the solution to lines beyond Vehicle, Property and Life.
- **Advanced claims automation:** add automated claim assessment and escalation based on claim characteristics.
- **Enhanced analytics:** build advanced dashboards and reports for policy, premium and claims analysis.
- **Customer experience:** add customer-facing tracking of policy and claim status.
- **Mobile access:** let agents and claims users reach key processes from mobile devices.
- **AI assistance:** explore AI help for agents and adjusters with policy and claim tasks.
- **Quality and configuration:** add PremiumCalculatorTest, move the threshold, queue Ids and premium rates into Custom Metadata, and fix the Days Open and policy type issues.
- **Deployment:** promote the build through sandboxes using Change Sets or Salesforce DX, with at least 75% org-wide Apex coverage before release.

11. Appendix

Ref	Item	Description
A	Custom objects	Policy__c, Claim__c (with Contact and User lookups)
B	Record Types	Policy: Auto, Property, Life. Claim: Accident, Property, Life
C	Automation	AutoQuotingFlow (Screen), claim routing flow, Submission Automation Flow, Claim Approver Screen Flow, Claim Policy Holder State Update flow; High Value Claim Approval process
D	Apex	PremiumCalculator, ClaimsAdjusterController
E	LWC	claimsDashboardLwc (Claims Dashboard), claimTileLwc (Claim Tile)
F	Validation	VIN_Must_Be_17_Characters on Policy
G	Security	Permission sets Insurance Agent Access, Claims Adjuster Access, Claims Manager Access; state-based sharing rules (CA, TX)
H	Testing	ClaimsAdjusterControllerTest
I	Reporting	Claims Dashboard and Salesforce Reports and Dashboards

12. Demo Link

Demo video:

https://drive.google.com/file/d/1zxcopMmCaDfqXX7OcunYS75uox_Xg5Fr/view?usp=drive_link

Project documentation:

https://drive.google.com/file/d/1qr6ufUQU63X3B343FyEerAIauAn5WXT3/view?usp=drive_link